

Lab Assignment 2

JDBC CRUD operations with PreparedStatement

1. Title

Implementation of CRUD Operations using JDBC PreparedStatement

2. Aim

To develop Java programs that perform Create, Read, Update, and Delete (CRUD) operations on a MySQL database using the PreparedStatement interface.

3. Learning Objectives

- Establish database connectivity using JDBC
- Use **PreparedStatement** to execute parameterized SQL queries
- Perform **INSERT, SELECT, UPDATE, DELETE** operations
- Prevent SQL Injection using PreparedStatement
- Handle SQL Exceptions effectively

4. Software Requirements

- JDK 8 or above
- Eclipse IDE / IntelliJ / NetBeans
- MySQL Server
- MySQL Connector/J Driver
- Database Name: **college**
- Table Name: **student**

5. Database Setup

Step 1: Create Database

```
CREATE DATABASE exp;  
USE college;
```

Step 2: Create Table

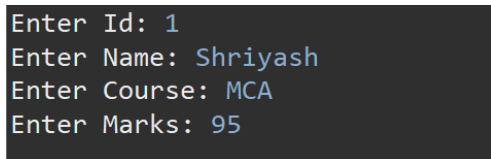
```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    course VARCHAR(30),  
    marks INT  
);
```

2.1 – Write a Java console program using PreparedStatement to insert a student record.

Output:

```
package com.exp2;
import java.sql.*;
import java.util.Scanner;
public class insertData {
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/exp"
        ,"root"
        ,"root");
        Scanner sc = new Scanner(System.in);
        PreparedStatement psmt = conn.prepareStatement(
            "INSERT INTO STUDENT (id, name, course, marks) VALUES
(? , ? , ? , ?)");
        System.out.print("Enter Id: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Course: ");
        String course = sc.nextLine();
        System.out.print("Enter Marks: ");
        int marks = sc.nextInt();
        psmt.setInt(1, id);
        psmt.setString(2, name);
        psmt.setString(3, course);
        psmt.setInt(4, marks);
        psmt.executeUpdate();
        sc.close();
        conn.close();
    }
}
```

Output:



```
Enter Id: 1
Enter Name: Shriyash
Enter Course: MCA
Enter Marks: 95
```

2.2.: Write a program to retrieve a student record by ID using PreparedStatement.

Code:

```
package com.exp2;

import java.sql.Connection;
import java.sql.DriverManager;
import java.util.Scanner;
import java.sql.*;
import java.util.Scanner;
public class retrieveData{
    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/exp"
        , "root"
        , "root");
        PreparedStatement psmt = conn.prepareStatement(
            "SELECT * FROM STUDENT WHERE ID=?"
        );
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        psmt.setInt(1, id);
        ResultSet rs = psmt.executeQuery();
        if (rs.next()) {
            System.out.println("\nID: " + rs.getInt("id"));
            System.out.println("Name: " + rs.getString("name"));
            System.out.println("Course: " + rs.getString("course"));
            System.out.println("Marks: " + rs.getInt("marks"));
        }
    }
}
```

Output:

```
Enter ID: 1

ID: 1
Name: Shriyash
Course: AI/ML
Marks: 90
```

2.3.:Write a program to update marks of a student using PreparedStatement.

Code:

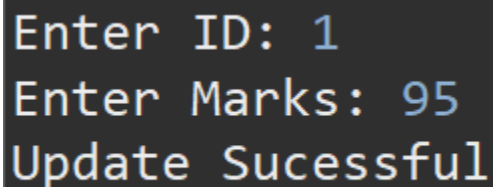
```
package com.exp2;
import java.sql.*;
import java.util.Scanner;

public class updateData {

    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/exp"
        ,"root"
        ,"root");

        PreparedStatement psmt = conn.prepareStatement(
            "UPDATE STUDENT SET marks = ? WHERE id = ?"
        );
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        psmt.setInt(2, id);
        System.out.print("Enter Marks: ");
        int marks = sc.nextInt();
        psmt.setInt(1, marks);
        psmt.executeUpdate();
        System.out.println("Update Sucessful");
    }
}
```

Output:

A screenshot of a terminal window with a dark background. It shows the output of the Java program: 'Enter ID: 1', 'Enter Marks: 95', and 'Update Sucessful'. The text is displayed in a light blue/cyan monospaced font.

```
Enter ID: 1
Enter Marks: 95
Update Sucessful
```

2.4.:Write a program to delete a student record using PreparedStatement.

Code:

```
package com.exp2;
import java.sql.*;
import java.util.Scanner;
public class deleteData {

    public static void main(String[] args) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/exp"
                            , "root"
                            , "root");

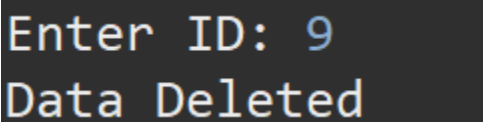
        PreparedStatement psmt = conn.prepareStatement(
            "DELETE FROM STUDENT WHERE ID = ?"
        );

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        sc.close();
        psmt.setInt(1, id);
        psmt.executeUpdate();
        conn.close();
        System.out.println("Data Deleted");

    }

}
```

Output:

A screenshot of a terminal window with a dark background. It shows the output of the Java program. The first line is "Enter ID: 9" where "9" is entered in blue. The second line is "Data Deleted" in white.

```
Enter ID: 9
Data Deleted
```

2.5.:Create a Swing application with:

- JTextField for id
- JTextField for name
- JTextField for course
- JTextField for marks
- Insert button

Code:

```
package com.exp2;
import javax.swing.*;
import java.awt.GridLayout;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class insertGUI {
    public static void main(String[] args) throws Exception {

        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn =
DriverManager.getConnection("jdbc:mysql://localhost:3306/exp"
                            , "root"
                            , "root");

        JFrame frame = new JFrame("Insert Data");
        frame.setSize(400, 300);
        frame.setLayout(new GridLayout(5, 2));

        JLabel idLabel = new JLabel("ID");
        JTextField idField = new JTextField();

        JLabel nameLabel = new JLabel("Name");
        JTextField nameField = new JTextField();

        JLabel courseLabel = new JLabel("Course");
        JTextField courseField = new JTextField();

        JLabel marksLabel = new JLabel("Marks");
        JTextField marksField = new JTextField();
```

```

JButton insertButton = new JButton("Insert");
frame.add(idLabel);
frame.add(idField);
frame.add(nameLabel);
frame.add(nameField);
frame.add(courseLabel);
frame.add(courseField);
frame.add(marksLabel);
frame.add(marksField);
frame.add(new JLabel());
frame.add(insertButton);
frame.setVisible(true);
insertButton.addActionListener(e->{
    try {
        PreparedStatement psmt = conn.prepareStatement(
            "INSERT INTO STUDENT VALUES
            (?, ?, ?, ?)"
        );
        psmt.setInt(1, Integer.parseInt(idField.getText()));
        psmt.setString(2, nameField.getText());
        psmt.setString(3, courseField.getText());
        psmt.setInt(4, Integer.parseInt(marksField.getText()));
        psmt.executeUpdate();
    } catch (SQLException e1) {
        e1.printStackTrace();
    }
});
}
}

```

Output:

Field Label	Value
ID	1
Name	Doraemon
Course	Robotics
Marks	95

Insert

2.6.:Create a Swing application that:

- Accepts student ID
- On clicking Search button
- Displays student details in JTextArea

Use PreparedStatement.

Code:

```
package com.exp2;
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*;
import java.sql.*;

public class retrieveGUI {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Search Student");
        frame.setSize(400, 250);
        frame.setLayout(new GridLayout(3, 2));
        JLabel idLabel = new JLabel("Enter Student ID:");
        JTextField idField = new JTextField();
        JButton searchButton = new JButton("Search");
        JTextArea resultArea = new JTextArea();
        resultArea.setEditable(false);
        frame.add(idLabel);
        frame.add(idField);
        frame.add(new JLabel());
        frame.add(searchButton);
        frame.add(new JLabel("Result:"));
        frame.add(new JScrollPane(resultArea));
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
        searchButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                try {
                    int id = Integer.parseInt(idField.getText().trim());

                    Class.forName("com.mysql.cj.jdbc.Driver");
                    Connection conn = DriverManager.getConnection(
                        "jdbc:mysql://localhost:3306/exp", "root", "root"
                    );

                    PreparedStatement psmt = conn.prepareStatement(
```



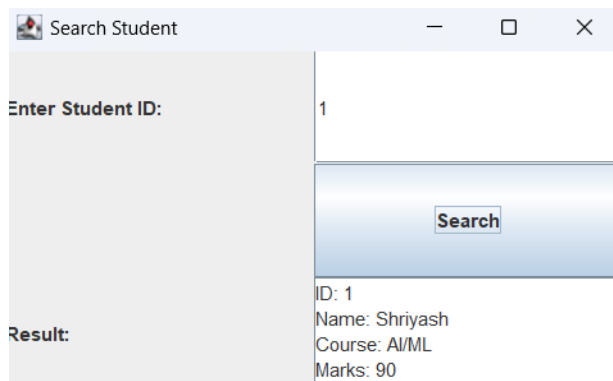
```
        "SELECT * FROM STUDENT WHERE ID=?"
    );
    pstmt.setInt(1, id);

    ResultSet rs = pstmt.executeQuery();

    if (rs.next()) {
        resultArea.setText(
            "ID: " + rs.getInt("id") + "\n" +
            "Name: " + rs.getString("name") + "\n" +
            "Course: " + rs.getString("course") + "\n" +
            "Marks: " + rs.getInt("marks")
        );
    } else {
        resultArea.setText("No record found for ID: " + id);
    }

    rs.close();
    pstmt.close();
    conn.close();
} catch (Exception ex) {
    JOptionPane.showMessageDialog(frame, "Error: " + ex.getMessage());
}
}
});
}
```

Output:



2.7.:Create a Swing application with buttons:

- Insert
- Search
- Update
- Delete
- Clear

All operations must use PreparedStatement.

Code:

```
package com.exp2;
```

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;
```

```
public class StudentCRUDApp {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Student CRUD Application");
        frame.setSize(500, 400);
        frame.setLayout(new FlowLayout());

        JLabel idLabel = new JLabel("ID");
        JTextField idField = new JTextField();

        JLabel nameLabel = new JLabel("Name");
        JTextField nameField = new JTextField();

        JLabel courseLabel = new JLabel("Course");
        JTextField courseField = new JTextField();

        JLabel marksLabel = new JLabel("Marks");
        JTextField marksField = new JTextField();

        JTextArea resultArea = new JTextArea();
        resultArea.setEditable(false);

        JButton insertBtn = new JButton("Insert");
        JButton searchBtn = new JButton("Search");
        JButton updateBtn = new JButton("Update");
        JButton deleteBtn = new JButton("Delete");
        JButton clearBtn = new JButton("Clear");
```

```
frame.add(idLabel); frame.add(idField);
frame.add(nameLabel); frame.add(nameField);
frame.add(courseLabel); frame.add(courseField);
frame.add(marksLabel); frame.add(marksField);
frame.add(insertBtn); frame.add(searchBtn);
frame.add(updateBtn); frame.add(deleteBtn);
frame.add(clearBtn); frame.add(new JScrollPane(resultArea));

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);

// Database connection details
String url = "jdbc:mysql://localhost:3306/exp";
String user = "root";
String pass = "root";

insertBtn.addActionListener(e -> {
    try (Connection conn = DriverManager.getConnection(url, user, pass)) {
        PreparedStatement psmt = conn.prepareStatement(
            "INSERT INTO STUDENT (id, name, course, marks) VALUES (?, ?, ?, ?)"
        );
        psmt.setInt(1, Integer.parseInt(idField.getText()));
        psmt.setString(2, nameField.getText());
        psmt.setString(3, courseField.getText());
        psmt.setInt(4, Integer.parseInt(marksField.getText()));
        psmt.executeUpdate();
        resultArea.setText("Record inserted successfully!");
    } catch (Exception ex) {
        resultArea.setText("Error: " + ex.getMessage());
    }
});

searchBtn.addActionListener(e -> {
    try (Connection conn = DriverManager.getConnection(url, user, pass)) {
        PreparedStatement psmt = conn.prepareStatement(
            "SELECT * FROM STUDENT WHERE id=?"
        );
    }
});
```

```
psmt.setInt(1, Integer.parseInt(idField.getText()));
ResultSet rs = psmt.executeQuery();
if (rs.next()) {
    resultArea.setText(
        "ID: " + rs.getInt("id") + "\n" +
        "Name: " + rs.getString("name") + "\n" +
        "Course: " + rs.getString("course") + "\n" +
        "Marks: " + rs.getInt("marks")
    );
} else {
    resultArea.setText("No record found.");
}
} catch (Exception ex) {
    resultArea.setText("Error: " + ex.getMessage());
}
});

updateBtn.addActionListener(e -> {
    try (Connection conn = DriverManager.getConnection(url, user, pass)) {
        PreparedStatement psmt = conn.prepareStatement(
            "UPDATE STUDENT SET name=?, course=?, marks=? WHERE id=?"
        );
        psmt.setString(1, nameField.getText());
        psmt.setString(2, courseField.getText());
        psmt.setInt(3, Integer.parseInt(marksField.getText()));
        psmt.setInt(4, Integer.parseInt(idField.getText()));
        int rows = psmt.executeUpdate();
        resultArea.setText(rows > 0 ? "Record updated successfully!" : "No record found.");
    } catch (Exception ex) {
        resultArea.setText("Error: " + ex.getMessage());
    }
});

deleteBtn.addActionListener(e -> {
    try (Connection conn = DriverManager.getConnection(url, user, pass)) {
        PreparedStatement psmt = conn.prepareStatement(
            "DELETE FROM STUDENT WHERE id=?"
        );
        psmt.setInt(1, Integer.parseInt(idField.getText()));
```

```
        int rows = psmt.executeUpdate();
        resultArea.setText(rows > 0 ? "Record deleted successfully!" : "No record found.");
    } catch (Exception ex) {
        resultArea.setText("Error: " + ex.getMessage());
    }
});

clearBtn.addActionListener(e -> {
    idField.setText("");
    nameField.setText("");
    courseField.setText("");
    marksField.setText("");
    resultArea.setText("");
});
}
```

Output:

Student CRUD Application

D: 1

Name:

Course:

Marks:

Insert

Search

Update

Delete

Clear

Result:

D: 1
Name: Shriyash
Course: AI/ML
Marks: 90

Conclusion

Thus, the Java programs for performing **CRUD operations using JDBC PreparedStatement** were successfully implemented. The application established database connectivity with the **MySQL database (college)** and performed **INSERT, SELECT, UPDATE, and DELETE** operations on the **student** table efficiently. The use of **PreparedStatement** helped execute parameterized queries securely, reduced the chances of **SQL injection attacks**, and improved performance. Additionally, exception handling ensured smooth execution of database operations. This practical enhanced understanding of **JDBC connectivity and real-time database interaction using Java**.